MFS 가변 파일 시스템

개요

MFS(Mutable File System)는 IPFS의 불변성을 유지하면서도 전통적인 파일 시스템처럼 파일과 디렉토리를 수정할 수 있게 해주는 메모리 내 가변 파일 시스템입니다. MFS는 파일 시스템의 변경사항을 추적하고, 필요할 때 IPFS DAG에 동기화하여 영구적으로 저장할 수 있습니다.

주요 개념

MFS의 핵심 구성 요소

- **Root**: MFS의 루트 디렉토리, 전체 파일 시스템의 진입점
- **Directory**: 가변 디렉토리, 자식 노드들을 관리
- **File**: 가변 파일, 읽기/쓰기 작업 지원
- **FileDescriptor**: 파일에 대한 핸들, 실제 I/O 작업 수행
- **Inode**: 파일과 디렉토리의 공통 추상화

MFS의 동작 원리

```
메모리 내 MFS 구조
├── Root (/)
| ├── Directory (docs/)
| | ├── File (readme.txt)
| | └── File (guide.md)
```

```
| └── File (config.json)
|
동기화 ↓
|
IPFS DAG 구조
├── Root Node (QmXXX...)
| ├── Directory Node (QmYYY...)
| | ├── File Node (QmZZZ...)
| | └── File Node (QmAAA...)
| └── File Node (QmBBB...)
```

기본 사용법

MFS 루트 생성

```
package main

import (
"context"
"fmt"

"github.com/ipfs/boxo/mfs"
"github.com/ipfs/boxo/ipld/merkledag"
```

```
"github.com/ipfs/boxo/ipld/unixfs"
"github.com/ipfs/go-datastore"
dssync "github.com/ipfs/go-datastore/sync"
blockstore "github.com/ipfs/go-ipfs-blockstore"
ipld "github.com/ipfs/go-ipld-format"
)

func createMFSRoot() (*mfs.Root, error) {
// 블록스토어와 DAG 서비스 설정
ds := dssync.MutexWrap(datastore.NewMapDatastore())
bs := blockstore.NewBlockstore(ds)
dagService := merkledag.NewDAGService(bs)

// 빈 디렉토리 노드 생성
emptyDir := unixfs.EmptyDirNode()
// DAG 서비스에 추가
ctx := context.Background()
err := dagService.Add(ctx, emptyDir)
if err != nil {
return nil, fmt.Errorf("빈 디렉토리 추가 실패: %v", err)
}

// MFS 루트 생성
```

```
root, err := mfs.NewRoot(ctx, dagService, emptyDir, nil)
if err != nil {
return nil, fmt.Errorf("MFS 루트 생성 실패: %v", err)
}

fmt.Printf("MFS 루트가 생성되었습니다. CID: %s\n", emptyDir.Cid())
return root, nil
}
```

디렉토리 생성

```
func createDirectory(root *mfs.Root, dirPath string) error {
ctx := context.Background()
// 디렉토리 생성
err := mfs.Mkdir(root, dirPath, mfs.MkdirOpts{
Mkparents: true, // 부모 디렉토리도 함께 생성
Flush: true, // 즉시 DAG에 동기화
})
if err != nil {
return fmt.Errorf("디렉토리 생성 실패: %v", err)
}

fmt.Printf("디렉토리가 생성되었습니다: %s\n", dirPath)
```

```
return nil
}
```

파일 생성 및 쓰기

```
package main

import (
"context"
"fmt"
"strings"

"github.com/ipfs/boxo/mfs"
)

func createAndWriteFile(root *mfs.Root, filePath string, content string) erro
ctx := context.Background()
// 파일 생성 (쓰기 모드로 열기)
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: false,
Write: true,
Create: true,
})
```

```
if err != nil {
return fmt.Errorf("파일 생성 실패: %v", err)
}
defer fd.Close()

// 파일에 내용 쓰기
reader := strings.NewReader(content)
_, err = fd.ReadFrom(reader)
if err != nil {
return fmt.Errorf("파일 쓰기 실패: %v", err)
}

// 변경사항을 DAG에 동기화
err = fd.Flush()
if err != nil {
return fmt.Errorf("파일 플러시 실패: %v", err)
}

fmt.Printf("파일이 생성되고 내용이 작성되었습니다: %s\n", filePath)
return nil
}
```

파일 읽기

```go
func readFile(root *mfs.Root, filePath string) (string, error) {
ctx := context.Background()
// 파일 열기 (읽기 모드)
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: true,
Write: false,
})
if err != nil {
return "", fmt.Errorf("파일 열기 실패: %v", err)
}
defer fd.Close()

// 파일 크기 확인
size, err := fd.Size()
if err != nil {
return "", fmt.Errorf("파일 크기 확인 실패: %v", err)
}

// 파일 내용 읽기
buffer := make([]byte, size)
n, err := fd.Read(buffer)
if err != nil && err != io.EOF {
```

```
return "", fmt.Errorf("파일 읽기 실패: %v", err)
}

content := string(buffer[:n])
fmt.Printf("파일 내용 (%s): %s\n", filePath, content)
return content, nil
}
```

파일 시스템 연산

파일 및 디렉토리 목록 조회

```
func listDirectory(root *mfs.Root, dirPath string) error {
ctx := context.Background()
// 디렉토리 조회
entries, err := mfs.ListDir(root, dirPath)
if err != nil {
return fmt.Errorf("디렉토리 목록 조회 실패: %v", err)
}

fmt.Printf("디렉토리 내용 (%s):\n", dirPath)
for _, entry := range entries {
// 항목 정보 출력
```

```
fmt.Printf("- %s", entry.Name)

if entry.Type == mfs.TDir {

fmt.Printf(" [디렉토리]")

} else if entry.Type == mfs.TFile {

fmt.Printf(" [파일]")

fmt.Printf(" (크기: %d)", entry.Size)

}

fmt.Printf(" CID: %s\n", entry.Hash)

}


return nil

}
```

파일 이동 및 이름 변경

```
func moveFile(root *mfs.Root, srcPath, dstPath string) error {

// 파일 이동

err := mfs.Mv(root, srcPath, dstPath)

if err != nil {

return fmt.Errorf("파일 이동 실패: %v", err)

}


fmt.Printf("파일이 이동되었습니다: %s -> %s\n", srcPath, dstPath)
```

```
return nil
}
```

파일 및 디렉토리 삭제

```
func removeFile(root *mfs.Root, filePath string) error {
// 파일 삭제
err := mfs.Rm(root, filePath, true) // true: 강제 삭제
if err != nil {
return fmt.Errorf("파일 삭제 실패: %v", err)
}

fmt.Printf("파일이 삭제되었습니다: %s\n", filePath)
return nil
}
```

파일 복사

```
func copyFile(root *mfs.Root, srcPath, dstPath string) error {
// 소스 파일 읽기
content, err := readFile(root, srcPath)
if err != nil {
return fmt.Errorf("소스 파일 읽기 실패: %v", err)
```

```
}

// 대상 파일에 쓰기
err = createAndWriteFile(root, dstPath, content)
if err != nil {
return fmt.Errorf("대상 파일 생성 실패: %v", err)
}

fmt.Printf("파일이 복사되었습니다: %s -> %s\n", srcPath, dstPath)
return nil
}
```

고급 활용

파일 스트리밍 쓰기

```
package main

import (
"context"
"fmt"
"io"
```

```
"github.com/ipfs/boxo/mfs"
)

func streamWriteFile(root *mfs.Root, filePath string, reader io.Reader) error
ctx := context.Background()
// 파일 생성
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: false,
Write: true,
Create: true,
})
if err != nil {
return fmt.Errorf("파일 생성 실패: %v", err)
}
defer fd.Close()

// 스트리밍 방식으로 데이터 쓰기
buffer := make([]byte, 64*1024) // 64KB 버퍼
totalWritten := int64(0)
for {
n, err := reader.Read(buffer)
if err == io.EOF {
break
```

```
}
if err != nil {
return fmt.Errorf("읽기 오류: %v", err)
}

written, err := fd.Write(buffer[:n])
if err != nil {
return fmt.Errorf("쓰기 오류: %v", err)
}
totalWritten += int64(written)
}

// 변경사항 동기화
err = fd.Flush()
if err != nil {
return fmt.Errorf("플러시 실패: %v", err)
}

fmt.Printf("스트리밍 쓰기 완료: %s (%d 바이트)\n", filePath, totalWritten)
return nil
}
```

파일 부분 수정

```
func modifyFileAtOffset(root *mfs.Root, filePath string, offset int64, newDat
// 파일 열기 (읽기/쓰기 모드)
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: true,
Write: true,
})
if err != nil {
return fmt.Errorf("파일 열기 실패: %v", err)
}
defer fd.Close()

// 지정된 오프셋으로 시크
_, err = fd.Seek(offset, io.SeekStart)
if err != nil {
return fmt.Errorf("시크 실패: %v", err)
}

// 새 데이터 쓰기
_, err = fd.Write(newData)
if err != nil {
return fmt.Errorf("쓰기 실패: %v", err)
}
```

```
// 변경사항 동기화
err = fd.Flush()
if err != nil {
return fmt.Errorf("플러시 실패: %v", err)
}

fmt.Printf("파일이 오프셋 %d에서 수정되었습니다: %s\n", offset, filePath)
return nil
}
```

파일 크기 조정 (Truncate)

```
func truncateFile(root *mfs.Root, filePath string, newSize int64) error {
// 파일 열기 (쓰기 모드)
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: false,
Write: true,
})
if err != nil {
return fmt.Errorf("파일 열기 실패: %v", err)
}
defer fd.Close()
```

```
// 파일 크기 조정
err = fd.Truncate(newSize)
if err != nil {
return fmt.Errorf("파일 크기 조정 실패: %v", err)
}

// 변경사항 동기화
err = fd.Flush()
if err != nil {
return fmt.Errorf("플러시 실패: %v", err)
}

fmt.Printf("파일 크기가 %d 바이트로 조정되었습니다: %s\n", newSize, filePath)
return nil
}
```

실시간 업데이트 처리

자동 동기화 설정

```
package main
```

```
import (
"context"
"time"

"github.com/ipfs/boxo/mfs"
)

func setupAutoSync(root *mfs.Root) {
ctx := context.Background()
// 자동 동기화 간격 설정 (5초마다)
ticker := time.NewTicker(5 * time.Second)
defer ticker.Stop()

go func() {
for {
select {
case <-ticker.C:
// 루트 플러시 (모든 변경사항 동기화)
err := root.Flush()
if err != nil {
fmt.Printf("자동 동기화 오류: %v\n", err)
} else {
fmt.Println("MFS 자동 동기화 완료")
```

```
}
case <-ctx.Done():
return
}
}
}()
}
```

변경사항 추적

```
func  trackChanges(root *mfs.Root) {
// 초기 루트 CID 저장
initialNode, err := root.GetDirectory().GetNode()
if  err != nil {
fmt.Printf("초기 노드 가져오기 실패: %v\n", err)
return
}
initialCID := initialNode.Cid()
fmt.Printf("초기 루트 CID: %s\n", initialCID)

// 변경사항 확인 함수
checkChanges := func() {
currentNode, err := root.GetDirectory().GetNode()
```

```
if err != nil {
fmt.Printf("현재 노드 가져오기 실패: %v\n", err)
return
}
currentCID := currentNode.Cid()
if !currentCID.Equals(initialCID) {
fmt.Printf("변경사항 감지됨! 새 CID: %s\n", currentCID)
initialCID = currentCID
}
}

// 주기적으로 변경사항 확인
ticker := time.NewTicker(2 * time.Second)
defer ticker.Stop()

for range ticker.C {
checkChanges()
}
}
```

배치 작업 처리

```
func batchOperations(root *mfs.Root) error {
```

```
ctx := context.Background()

// 여러 파일 생성 (배치 모드)

files := map[string]string{

"/batch/file1.txt": "첫 번째 파일 내용",

"/batch/file2.txt": "두 번째 파일 내용",

"/batch/file3.txt": "세 번째 파일 내용",

}



// 부모 디렉토리 생성

err := mfs.Mkdir(root, "/batch", mfs.MkdirOpts{

Mkparents: true,

Flush: false, // 개별 작업에서는 플러시하지 않음

})

if err != nil {

return fmt.Errorf("배치 디렉토리 생성 실패: %v", err)

}



// 모든 파일 생성

for filePath, content := range files {

err := createAndWriteFile(root, filePath, content)

if err != nil {

return fmt.Errorf("파일 생성 실패 (%s): %v", filePath, err)

}
```

```
}

// 모든 작업 완료 후 한 번에 동기화
err = root.Flush()
if err != nil {
return fmt.Errorf("배치 동기화 실패: %v", err)
}

fmt.Printf("배치 작업 완료: %d개 파일 생성\n", len(files))
return nil
}
```

설정 옵션

MFS 생성 옵션

```
func createMFSWithOptions() (*mfs.Root, error) {
ctx := context.Background()
// DAG 서비스 설정
ds := dssync.MutexWrap(datastore.NewMapDatastore())
bs := blockstore.NewBlockstore(ds)
dagService := merkledag.NewDAGService(bs)
```

```
// 빈 디렉토리 생성
emptyDir := unixfs.EmptyDirNode()
err := dagService.Add(ctx, emptyDir)
if err != nil {
return nil, err
}

// 콘텐츠 제공자 설정 (선택사항)
var provider routing.ContentProviding = nil // 또는 실제 제공자

// MFS 루트 생성 (제공자와 함께)
root, err := mfs.NewRoot(ctx, dagService, emptyDir, provider)
if err != nil {
return nil, err
}

return root, nil
}
```

파일 열기 옵션

```
func demonstrateFileFlags(root *mfs.Root, filePath string) error {
// 읽기 전용 모드
```

```
fdRead, err := mfs.Open(root, filePath, mfs.Flags{
Read: true,
Write: false,
})
if err == nil {
defer fdRead.Close()
fmt.Println("읽기 전용 모드로 파일 열기 성공")
}

// 쓰기 전용 모드 (파일 생성)
fdWrite, err := mfs.Open(root, filePath+"_new", mfs.Flags{
Read: false,
Write: true,
Create: true,
})
if err == nil {
defer fdWrite.Close()
fmt.Println("쓰기 전용 모드로 파일 생성 성공")
}

// 읽기/쓰기 모드
fdReadWrite, err := mfs.Open(root, filePath, mfs.Flags{
Read: true,
```

```
Write: true,
})
if err == nil {
defer fdReadWrite.Close()
fmt.Println("읽기/쓰기 모드로 파일 열기 성공")
}

return nil
}
```

모범 사례

1. 리소스 관리

```
func properResourceManagement(root *mfs.Root, filePath string) error {
// 파일 디스크립터는 반드시 닫아야 함
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: true,
Write: false,
})
if err != nil {
return err
}
```

```go
// defer를 사용하여 확실한 정리
defer func() {
if closeErr := fd.Close(); closeErr != nil {
fmt.Printf("파일 닫기 오류: %v\n", closeErr)
}
}()

// 파일 작업 수행
return processFile(fd)
}
```

2. 오류 처리

```go
func robustFileOperations(root *mfs.Root, filePath string) error {
// 파일 존재 여부 확인
_, err := mfs.Stat(root, filePath)
if err != nil {
if os.IsNotExist(err) {
fmt.Printf("파일이 존재하지 않습니다: %s\n", filePath)
return nil
}
return fmt.Errorf("파일 상태 확인 실패: %v", err)
}
```

```go
// 안전한 파일 열기
fd, err := mfs.Open(root, filePath, mfs.Flags{
Read: true,
Write: false,
})
if err != nil {
return fmt.Errorf("파일 열기 실패: %v", err)
}
defer fd.Close()

// 파일 크기 확인
size, err := fd.Size()
if err != nil {
return fmt.Errorf("파일 크기 확인 실패: %v", err)
}

// 큰 파일에 대한 특별 처리
if size > 100*1024*1024 { // 100MB
return fmt.Errorf("파일이 너무 큽니다: %d 바이트", size)
}

return nil
```

```
}
```

3. 성능 최적화

```
func optimizedBatchWrite(root *mfs.Root, files map[string][]byte) error {
// 플러시 지연을 위한 설정
for filePath, content := range files {
fd, err := mfs.Open(root, filePath, mfs.Flags{
Write: true,
Create: true,
})
if err != nil {
return err
}

// 내용 쓰기 (개별 플러시 없음)
_, err = fd.Write(content)
fd.Close() // 플러시 없이 닫기
if err != nil {
return err
}
}
```

```
// 모든 작업 완료 후 한 번에 플러시
return root.Flush()
}
```

문제 해결

일반적인 오류와 해결책

1. **"file closed" 오류**
   - 이미 닫힌 파일 디스크립터에 접근할 때 발생
   - 파일 디스크립터의 생명주기 관리 확인
2. **"file is read-only" 오류**
   - 읽기 전용으로 열린 파일에 쓰기 시도
   - 올바른 플래그로 파일 열기
3. **메모리 부족 오류**
   - 큰 파일 처리 시 발생
   - 스트리밍 방식 사용 또는 배치 크기 조정

성능 모니터링

```
func monitorMFSPerformance(root *mfs.Root) {
// 루트 노드 크기 모니터링
node, err := root.GetDirectory().GetNode()
if err != nil {
```

```
fmt.Printf("노드 가져오기 실패: %v\n", err)

return

}



// DAG 통계 출력

fmt.Printf("루트 CID: %s\n", node.Cid())

fmt.Printf("노드 크기: %d 바이트\n", len(node.RawData()))

// 링크 수 확인

links := node.Links()

fmt.Printf("자식 노드 수: %d\n", len(links))

for  i, link := range  links {

fmt.Printf(" 링크 %d: %s (크기: %d)\n",

i, link.Name, link.Size)

}

}
```

MFS는 IPFS의 불변성과 전통적인 파일 시스템의 가변성을 결합한 강력한 도구입니다. 적절한 동기화 전략과 리소스 관리를 통해 효율적인 파일 시스템 작업이 가능합니다.